



南开大学
Nankai University

HE from LWE: Conceptually-Simpler, Asymptotically- Faster, Attribute-Based

汇报人：张浩然



汇报 提纲

CONTENTS

论文背景

方案预览

基础加密方案

基于身份与属性基的 FHE 方案

FHE 发展历程

第一代

mod- p 整数
算术电路

关键词:

- Gentry blueprint
- squashing
- bootstrapping

代表论文:

Gentry 2009

第二代

mod- p 整数
算术电路

关键词:

- LWE / $RLWE$
- modulus switching
- SIMD / packing

代表论文:

BV 2011

BGV 2011

BFV 2012

第三代

bits
Boolean 电路

关键词:

- GSW 路线
- 快速
bootstrapping
- bit-wise 计算

代表论文:

GSW 2013

FHEW 2015

TFHE 2016

第四代

实数 / 复数
近似算术

关键词:

- CKKS
- 近似同态计算
- 面向机器学习

代表论文:

CKKS 2017

GSW 标志着 FHE 从“依赖重线性化等修复机制维持乘法”的旧路线，转向了“以自然矩阵运算直接实现同态乘法”的新范式。

为什么 GSW 的乘法思路不同？

BGV / BFV 的做法

乘后复杂，再修回去

1. 乘法后密文结构变复杂；
2. 再通过 **密钥切换 / 重线性化**把复杂结构标准化。

因此求值者通常还需要一份 **evaluation key**（求值密钥），来完成这一步“修复”。

它更像一类**算术电路型 FHE**：

重点是让密文在多项式环里持续可算，因此较依赖 **重线性化、模交换**等维护机制。

GSW 的做法

从结构上让乘法直接成立

GSW 试图绕开“乘后修复”这件事。它把密文定义成矩阵，并让解密关系直接满足

$$Cv \approx \mu v.$$

若 $C_1v \approx \mu_1v$, $C_2v \approx \mu_2v$, 则自然有 $(C_1C_2)v \approx \mu_1\mu_2v$.

因此它**不需要额外的 evaluation key** 去把乘法结果重新压回标准形状。

所以：**同态乘法本身就是矩阵乘法**。它一开始就把密文设计成对乘法闭合的结构。

成果预览

结果一：更简单的 FHE

GSW 基于 **LWE** 构造全同态加密，并将密文改写为矩阵形式，使同态加法与乘法都对应为自然的矩阵运算，从而让方案描述更统一、乘法机制更直观。

结果二：不需要 evaluation key

GSW 不再依赖重线性化矩阵来“修复”密文结构，因此求值者只需知道系统基本参数，而不再需要用户专属的求值密钥。

结果三：首次得到 IBFHE

GSW 给出了一个编译思路：只要底层是满足条件的 **LWE 型 IBE**，就可以进一步得到 **IBFHE**，从而支持同一身份下密文的同态处理。

结果四：可扩展到 ABFHE

类似地，这套方法还可以推广到属性加密场景，把已有的 **ABE for circuits** 编译为 **attribute-based FHE**，使同一 index 下的密文能够继续进行同态计算。

汇报 提纲

CONTENTS

论文背景

方案预览

基础加密方案

基于身份与属性基的 FHE 方案

无误差版本同态运算

先暂时忽略噪声，把 GSW 理解成“矩阵的特征值编码”

设密文是矩阵 C ，秘密向量是 v ，消息是小整数 μ 。若理想情况下有

$$Cv = \mu v,$$

那么 v 就是 C 的特征向量， μ 就是对应的特征值。

加法为什么自然？

若

$$C_1 v = \mu_1 v, \quad C_2 v = \mu_2 v,$$

则

$$(C_1 + C_2)v = (\mu_1 + \mu_2)v.$$

所以：

矩阵加法对应消息加法。

乘法为什么自然？

同样有

$$C_1 C_2 v = C_1 (\mu_2 v) = \mu_2 (C_1 v) = \mu_1 \mu_2 v.$$

所以：

矩阵乘法对应消息乘法。

如果真能做到这一点，那么同态加法、乘法就都变成了自然的矩阵运算。

带误差版本同态运算

真实方案不是 $Cv = \mu v$ ，而是 **approximate eigenvector**

在真正的 GSW 中，关系变成 $Cv = \mu v + e$ ，其中 e 是小误差。
也就是说， v 不再是精确特征向量，而是近似特征向量。

为什么必须有误差？

因为方案要建立在
LWE 上。

没有误差，就没有 LWE 型
安全性。

所以 GSW 不是无噪声方案，
而是把噪声控制在
“不会破坏解密”的范围内。

乘法后会发生什么？

若 $C_i v = \mu_i v + e_i$ ，则

$$\begin{aligned} C_1 C_2 v &= \mu_1 \mu_2 v + \mu_2 e_1 + C_1 e_2 \\ &= \mu_1 \mu_2 v + \text{small.} \end{aligned}$$

所以真正的难点不是乘法本
身，而是误差不能长太快。

GSW 怎么处理？

作者进一步引入
Flatten，把密文系数压回小
范围，
同时不改变与 v 的乘积关系。
再配合 NAND 与小消息不变
量，最终把“自然矩阵运算”
推进到 leveled FHE。

B-有界密文

若密文满足 $Cv = \mu v + e$, 则解密能否继续正确, 关键在于: 误差 e 会不会迅速变大。

定义: B-有界密文

设 C_1, C_2 是 B-有界密文, 即

$$|\mu_i| \leq B, \quad |(C_i)_{jk}| \leq B, \quad |(e_i)_j| \leq B.$$

也就是说, 消息、密文系数与误差系数的绝对值都不超过同一个上界 B 。

加法: 增长较温和

令 $C_+ = C_1 + C_2$, 则

$$C_+ v = (\mu_1 + \mu_2)v + (e_1 + e_2).$$

因此

C_+ 是 $2B$ -有界的。

直观上:

加法只是把误差叠加起来。

乘法: 增长明显更快

令 $C_\times = C_1 C_2$, 则

$$C_\times v = \mu_1 \mu_2 v + \mu_2 e_1 + C_1 e_2.$$

因此

$C_\times$ 是 $(N+1)B^2$ -有界的。

直观上:

乘法会让误差近似按平方级恶化。

SWHE 部分同态加密

若继续增加乘法深度，误差会大致沿着

$$B, B^2, B^4, \dots, B^{2^L}$$

这样快速恶化。

一旦误差逐渐接近模数 q ，它就不再是“小扰动”，解密所依赖的舍入条件也会失效。
因此，到目前为止，GSW 得到的还只是一个

Somewhat Homomorphic Encryption

即 **SWHE** 方案。

它可以支持：

对数深度电路 或 有限次数多项式

强有界密文

若密文 C 满足 $Cv = \mu v + e$, 且消息 μ 与矩阵 C 的各个系数都至多为 1, 误差向量 e 的各个系数都至多为 B , 则称 C 是 B -强有界的。

对两个 B -强有界密文 C_1, C_2 做 NAND: $C_3 \leftarrow I_N - C_1 C_2$.

新消息仍在 $\{0, 1\}$, 新误差至多为 $(N + 1)B$.

如果再通过 Flatten 让 C_3 的矩阵系数重新压回到不超过 1, 那么“强有界性”就在每一层 NAND 后都能保持。

于是, 对深度为 L 的电路, 误差可控制为

$$(N + 1)^L B.$$

密文展平 (Flatten)

在 GSW 中, 做完一层 NAND 后, 先得到中间密文 $C_3 \leftarrow I_N - C_1 C_2$. 完成布尔运算, 但 C_3 的矩阵系数未必很小, 因而“强有界性”可能被破坏。

Flatten 的定义

$$\text{Flatten}(C) = \text{BitDecomp}(\text{BitDecomp}^{-1}(C)).$$

它会把矩阵重新写成 0/1 小系数形式, 但不改变它与秘密向量 $v = \text{Powersof2}(s)$ 的乘积关系:

$$\text{Flatten}(C) v = C v.$$

经过 Flatten 之后

- 消息对应关系保持不变;
- 误差分析保持不变;
- 密文系数重新压回小范围;
- 强有界性得以逐层保持。

密钥生成

选取参数 $q, n, m = O(n \log q)$, 生成公开矩阵

$$A \in \mathbb{Z}_q^{m \times (n+1)},$$

并存在秘密向量

$$s \in \mathbb{Z}_q^{n+1}, \quad e = As = \text{small},$$

其中 s 的第一项为 1。

GSW 不直接把 s 作为秘密向量使用, 而是进一步定义

$$v = \text{Powersof2}(s), \quad N = (n+1)\ell, \quad \ell = \lceil \log_2 q \rceil + 1.$$

这样做的作用是两点:

- v 具备“按 2 的幂展开”的结构, 便于后续配合 BitDecomp 与 Flatten;
- v 中天然包含大坐标, 便于解密时通过“除法 + 舍入”恢复消息。

加密

对消息 $\mu \in \mathbb{Z}_q$, 先随机采样

$$R \in \{0, 1\}^{N \times m},$$

再输出密文

$$C = \text{Flatten}(\mu I_N + \text{BitDecomp}(RA)).$$

这一步可以分成两层理解:

- RA : 把公开的 LWE 样本重新随机组合, 得到“很多个加密 0”;
- μI_N : 把消息 μ 直接嵌入矩阵, 对应后续的“近似特征值”关系。

由于 Flatten 不改变与 v 的乘积关系, 因此有

$$Cv = \mu v + \text{BitDecomp}(RA) v = \mu v + RAs = \mu v + Re.$$

所以密文 C 满足 GSW 最核心的形式:

$$Cv = \mu v + \text{small}.$$

汇报 提纲

—
CONTENTS

论文背景

方案预览

基础加密方案

基于身份与属性基的 FHE 方案

Step 1 — Setup

先固定整套 LWE 参数与矩阵维度

GSW 在这一步并不直接生成密钥，而是先固定后续所有算法都要共用的系统参数：
 $\text{params} = (n, q, \chi, m)$.

q : 模数, n : LWE 维度, χ : 误差分布, $m = O(n \log q)$.

接着定义两个很关键的派生参数：

$$\ell = \lceil \log q \rceil + 1, \quad N = (n + 1)\ell.$$

- ℓ 表示把模 q 下元素按二进制展开所需的位数；
- N 是后续矩阵密文与扩展秘密向量的维度；
- 之所以从 $n + 1$ 扩到 $N = (n + 1)\ell$ ，是因为后面要配合 **BitDecomp** / **Powersof2** / **Flatten** 这些操作。

Step 2 — SecretKeyGen

把普通秘密向量变成“近似特征向量”

先均匀采样 $t \leftarrow \mathbb{Z}_q^n$, 然后定义秘密钥为 $s = (1, -t_1, \dots, -t_n) \in \mathbb{Z}_q^{n+1}$.

这里首坐标固定为 1, 不是随便写的, 而是为了和 LWE 的矩阵表示保持一致。后面公钥矩阵 A 会满足 $A \cdot s = e$, 其中 e 是小误差向量。

GSW 真正用于解密的还不是 s 本身, 而是 $v = \text{Powersof2}(s)$.

s 中每个分量都会按 $(1, 2, 4, \dots, 2^{\ell-1})$ 这一组权重复制展开, 得到一个更长的向量 v 。

这样设计的作用有两层:

- v 与 BitDecomp 正好是配套的, 可满足

$$\langle \text{BitDecomp}(a), \text{Powersof2}(b) \rangle = \langle a, b \rangle;$$

- v 中天然会出现足够大的坐标, 后面解密可以通过某一行内积再做舍入来恢复消息。

Step 3 — PublicKeyGen

把 LWE 样本组织成一个公开矩阵 A

先均匀生成 $B \leftarrow \mathbb{Z}_q^{m \times n}$, $e \leftarrow \chi^m$, 再令 $b = B \cdot t + e$.

随后把 b 作为第一列, B 的各列接在后面, 组成公钥矩阵

$$A = (b \mid B) \in \mathbb{Z}_q^{m \times (n+1)}.$$

这一写法最关键的地方是: $A \cdot s = e$. 因为 $A \cdot s = b - Bt = (Bt + e) - Bt = e$.

此式说明:

- 如果把 A 看成公钥, 那么它对秘密向量 s 的作用结果不是 0, 而是一个小误差;
- 所以 A 的每一行, 本质上都对应一个 Regev 型 LWE 样本;
- 后面只要随机组合这些行, 就能得到“加密 0”的结构。

Step 4 — Enc

将“加密 0”与消息 μ 组合成矩阵密文

对消息 $\mu \in \mathbb{Z}_q$, 随机采样

$$R \in \{0, 1\}^{N \times m},$$

输出密文

$$C = \text{Flatten}(\mu I_N + \text{BitDecomp}(RA)) \in \mathbb{Z}_q^{N \times N}.$$

第一层: RA

A 的每一行都是 LWE 样本, 所以 RA 是它们的随机组合。

由 $As = e$ 可得 $(RA)s = Re$.

因此 RA 可看成

“加密 0” (不改变真正消息, 但又能提供随机性和安全性)。

第二层: μI_N

把消息写成 μI_N 加到矩阵里。
这样密文满足

$$Cv \approx \mu v,$$

消息 μ 直接表现为
“特征值”。

第三层: Flatten

Flatten 把系数压小,
便于后续同态乘法。

关键性质:

$$\text{Flatten}(C)v = Cv.$$

因此它会整理密文形状,
但不破坏解密关系。

$C = \text{Flatten}(\mu I_N + \text{BitDecomp}(RA))$: 先加入消息, 再混入“加密 0”, 最后整理成标准矩阵密文。

从 $Cv = \mu v + \text{small}$ 中读出消息

只考虑比特消息 $\mu \in \{0, 1\}$. 由加密正确性可得 $Cv = \mu v + Re$, $v = \text{Powersof2}(s)$.

1. 选坐标

在 v 的前 ℓ 个坐标中, 取一个满足 $v_i \in (q/4, q/2]$ 的大坐标。

2. 做内积

记 C_i 为 C 的第 i 行, 计算 $x_i = \langle C_i, v \rangle$. 则 $x_i = \mu v_i + \langle R_i, e \rangle$.

3. 恢复

$\mu = \left\lfloor \frac{x_i}{v_i} \right\rfloor$. 若 $|\langle R_i, e \rangle| < q/8$, 则 $\left| \frac{x_i}{v_i} - \mu \right| < \frac{1}{2}$, 因此可以正确解密。

若 v_i 过小, 则主项 μv_i 不够突出, 误差在归一化后会占据过大比例, 从而使舍入恢复 μ 变得不稳定。

Step 6 — MPDec

将比特解密扩展到一般消息 $\mu \in \mathbb{Z}_q$

前面的 **Dec** 只适合恢复小消息，如 $\mu \in \{0, 1\}$ 。若希望恢复一般消息，GSW 引入 **MPDec**：利用多个坐标，按位恢复 μ 。

基本条件

当 $q = 2^{\ell-1}$ 时，
 $v = \text{Powersof2}(s)$ 的前 $\ell - 1$
个坐标正好是 $1, 2, \dots, 2^{\ell-2}$ 。
因此它天然带有
二进制权重结构。

读出什么信息？

若仍有 $Cv = \mu v + \text{small}$ ，则
前 $\ell - 1$ 个坐标近似为
 $\mu \cdot (1, 2, \dots, 2^{\ell-2}) + \text{small}$ 。
也就是说，这一段坐标里包
含了消息 μ 的逐位信息。

如何恢复？

- 先判断 μ 的奇偶性，恢复最低位 b_0 ；
- 减去 b_0 的贡献，再除去一个 2 的因子；
- 继续恢复下一位 b_1 ；
- 依次递推，直到恢复整个 μ 。

MPDec 说明 GSW 的基本框架可支持一般消息；但后文分析同态 NAND 时，作者仍主要使用比特消息 $\{0, 1\}$ ，因为这样最直观、误差分析也最干净。

关键引理： (A, RA) 与均匀分布不可区分

Lemma 1 (implicit in Regev)

若参数

$$\text{params} = (n, q, \chi, m)$$

满足 $\text{LWE}_{n,q,\chi}$ 假设，且

$$m = O(n \log q),$$

则由算法生成的联合分布

$$(A, RA)$$

与均匀分布计算上不可区分；论文中进一步指出， $m > 2n \log q$ 就已经足够。

由引理推出基本加密方案安全性

由加密算法,

$$\text{BitDecomp}^{-1}(C) = \mu G + RA,$$

因此密文的随机性核心就在

$$\mu G + RA.$$

又因为上面的引理说明 RA 看起来像均匀随机矩阵, 所以在 $\mu = 0$ 与 $\mu = 1$ 两种情况下,

$$\mu G + RA$$

都难以区分, 进而 C 也不泄露消息 μ 。

基本加密方案安全性直接归约到 **LWE**;
也可借助已有经典归约把它联系到 **GapSVP**。

核心不变量：运算后仍尽量保持 $Cv \approx \mu v$

若密文 C 加密消息 μ ，则满足 $Cv = \mu v + e$ 。GSW 的目标：运算后，新密文仍保持此形状。

1. Add：对应矩阵加法

$$\text{Add}(C_1, C_2) = \text{Flatten}(C_1 + C_2)$$

若 $C_i v = \mu_i v + e_i$ ，则

$$(C_1 + C_2)v = (\mu_1 + \mu_2)v + (e_1 + e_2).$$

直观上：消息相加，误差也相加

2. Mult：对应矩阵乘法

$$\text{Mult}(C_1, C_2) = \text{Flatten}(C_1 C_2)$$

由 $C_2 v = \mu_2 v + e_2$ 可得

$$C_1 C_2 v = \mu_1 \mu_2 v + \mu_2 e_1 + C_1 e_2.$$

直观上：消息直接相乘 (核心)

3. 另外两个辅助操作

$$\text{MultConst}(C, \alpha) = \text{Flatten}(M_\alpha C), \quad M_\alpha = \text{Flatten}(\alpha I_N)$$

用于密文 $\times$ 常数，比“重复加法”更稳、更省噪声。

为什么论文重点分析 NAND ? 因为它能把消息始终限制在 $\{0, 1\}$

普通乘法的一个问题是：新误差里会出现 $\mu_2 e_1 + C_1 e_2$, 因此误差增长会依赖中间消息大小。
论文的处理方法是：转向布尔电路，只保留 **NAND**。

NAND 的定义

对 $\mu_1, \mu_2 \in \{0, 1\}$, 定义 $\text{NAND}(C_1, C_2) = \text{Flatten}(I_N - C_1 C_2)$.

于是 $\text{NAND}(C_1, C_2)v = (1 - \mu_1 \mu_2)v - \mu_2 e_1 - C_1 e_2$.

这一步有两个关键好处：

- 输出消息仍在 $\{0, 1\}$ 中，小消息不变量被保持住；
- 因为 $\mu_2 \in \{0, 1\}$, 每层误差最多放大约 $N + 1$ 倍。

若初始误差为 B , 深度为 L 的 NAND 电路后, 误差约增至 $(N + 1)^L B$ 。

只要 $q/B > 8(N + 1)^L$, 即可正确解密, 因此 GSW 直接得到 leveled FHE。

参数、性能与优化

参数

深度为 L 的 NAND 电路后，
误差约为

$$(N + 1)^L B.$$

只要

$$q/B > 8(N + 1)^L,$$

解密仍然正确。

性能

深度为 L 的 NAND 电路中，
每个门的主要成本是矩阵乘法：

$\tilde{O}(nL)^\omega$ field operations per gate,

相比此前 LWE 型方案至少

$\tilde{O}(n^3 L)$ field operations per gate,

GSW 的渐近复杂度更优。

优化

Bootstrapping 可把每门开销
做成与 L 无关。

但它会重新引入加密密钥等
“非自然操作”。

因此 GSW 的核心优势仍是：

自然矩阵运算

而非 bootstrapping。

GSW 用更自然的同态结构，换来了更好的渐近性能；而 bootstrapping 只是可选优化。

汇报 提纲

CONTENTS

论文背景

方案预览

基础加密方案

基于身份与属性基的 FHE 方案

核心问题

为什么旧方案难以做到真正的 IBFHE / ABFHE ?

旧路线的障碍

以往 FHE 在同态求值时，通常还需要用户专属的 **evaluation key**。

这份 key：

- 不是 identity-based 的；
- 不能仅由身份 ID 非交互地产生；
- 会破坏 IBE 最重要的 **non-interactivity**。

所以此前最多只能得到弱形式 IBFHE。

GSW 的突破

GSW 方案没有 **evaluation key**，求值者只需知道一些基本参数（如 n, m, ℓ ）。

因此进一步得到：

- 第一个真正的 IBFHE；
- 并可用类似思路，将 GVW 的 **ABE for circuits** 编译为 ABFHE。

也就是说，第四部分真正利用的，正是“**无 evaluation key**”这一点。

GVW: Gorbunov、Vaikuntanathan 和 Wee 在 2013 年 STOC 上发表的 *Attribute-based encryption for circuits*；GSW 第四部分就是基于这篇 ABE for circuits 工作进一步得到 ABFHE。

基于身份与属性基同态加密

一、基于身份的同态加密 (IBHE)

它是在传统 IBE 上加入同态求值算法：

$$c \leftarrow \text{Eval}(\text{MPK}, ID, f, c_1, \dots, c_t),$$

其中各密文都加密在同一身份 ID 下 (本文只讨论 single-identity setting)，解密后得到 $\text{Dec}(sk_{ID}, c) = f(m_1, \dots, m_t)$ 。也就是说，身份既决定访问权限，也决定同态计算的归属对象。

二、基于属性的同态加密 (ABHE)

它是在传统 ABE 上加入同态求值算法。若密文以同一 index x 加密，则可计算

$$c \leftarrow \text{Eval}(\text{MPK}, x, f, c_1, \dots, c_t),$$

并由满足关系 $R(x, y) = 1$ 的密钥 sk_y 解密得到 $f(m_1, \dots, m_t)$ 。

在 ABHE 中，同态计算不会打乱原来的访问规则：密文允许所有满足某种属性关系的人来解。

IBHE / ABHE 的目标形式

这里要的不是普通 IBE/ABE，而是“可同态计算”的版本

1. Identity-based HE

对同一身份 ID 下的密文 $c_i \leftarrow \text{Enc}(\text{MPK}, ID, m_i)$ ，同态求值算法应输出 $c \leftarrow \text{Eval}(\text{MPK}, ID, f, c_1, \dots, c_t)$ ，并满足 $\text{Dec}(sk_{ID}, c) = f(m_1, \dots, m_t)$ 。

2. Attribute-based HE

对同一 index x 下的密文 $c_i \leftarrow \text{Enc}(\text{MPK}, x, m_i)$ ，应能计算 $c \leftarrow \text{Eval}(\text{MPK}, x, f, c_1, \dots, c_t)$ ，并由任意满足 $R(x, y) = 1$ 的密钥 sk_y 解密，得到 $f(m_1, \dots, m_t)$ 。

核心不是“谁能加密”，而是“同态后仍保持原有访问控制结构”

编译条件

Property 1: 向量型密文与密钥

身份 ID 的解密密钥与密文都写成向量 $s_{ID}, c_{ID} \in \mathbb{Z}_q^{n'}$ 且 s_{ID} 的第一坐标为 1。

Property 2: 加密 0 时存在“小点积”

若 c_{ID} 加密 0, 则 $\langle c_{ID}, s_{ID} \rangle$ 是 small。

Property 3: 安全性仍然来自 LWE

加密 0 的密文应与 $\mathbb{Z}_q$ 上均匀向量不可区分。

(因为如果“加密 0”太有规律, 别人一眼就能认出来, 那你后面即使把消息 μ 加进去, 也藏不住。)

这三个条件, 正好对应第 3 部分矩阵 FHE 所需要的“向量 + 小误差 + LWE 隐藏性”

把“同一身份下的加密 0”组织成 GSW 型矩阵密文

Step 1. 继承底层 IBE

IBFHE 直接使用底层 IBE 的 Setup, KeyGen, 同时补入第 3 部分 FHE 所需参数 m, ℓ, N 。

Step 2. 加密消息 $\mu \in \{0, 1\}$ 到身份 ID

先调用底层 IBE 的 $\text{Enc}(MPK, ID, 0)$ 生成 N 个“加密 0”的行向量, 组成矩阵 $\bar{C}_{ID}$, 再输出

$$C_{ID} = \text{Flatten}(\mu I_N + \text{BitDecomp}(\bar{C}_{ID})).$$

Step 3. 解密与同态求值

令 $v_{ID} = \text{Powersof2}(s_{ID})$, 解密时直接运行 $\text{Dec}(v_{ID}, C_{ID})$;

同态运算仍使用 Add, Mult, NAND.

$$C_{ID} \cdot v_{ID} = \mu v_{ID} + \bar{C}_{ID} \cdot s_{ID} = \mu v_{ID} + \text{small}$$

ABFHE 推广

ABFHE 的处理方式与 IBFHE 类似，但要先重解释 GVW 的解密过程

在 GVW 的 ABE 中，若 $R(x, y) = 1$ ，则密钥 sk_y 能解开 index 为 x 的密文。

作者把这一解密过程重新理解为：由 (x, y) 导出一个 **sub-key** $s_{x,y}$ ，并满足与 IBE 类似的性质： $\langle c_x, s_{x,y} \rangle = \text{small}$ （当 c_x 加密 0）。

于是同样可以套用前面的 compiler：

$$\text{GVW ABE} \implies \text{ABFHE}$$

同一 index x 下的密文可同态处理；原本有权解密这些密文的用户，仍能解出同态结果。

这一节在全文中的真正作用

- 说明“无 evaluation key”不只是结构更简洁；
- 它直接解决了 IBFHE 长期未解的问题；
- 也把第 3 部分 FHE 提升为一个可复用的 **compiler 框架**。

Thanks!